CAPSTONE REPOSITORY / RESEARCH COLLECTION

GridWise: Solar Microgrid Dispatch Under Uncertainty

Graduation-project demonstration dossier

King Khalid University - CE - 2026–2027

Authored demonstration team: Rakan Ali Alqahtani, Badr Ali Alqahtani, Sultan Ali Alqahtani, Ibrahim Ali Alqahtani

A small energy store must trade grid draw against battery wear while handling a bounded uncertain solar input. Dispatch decisions require an explicit feasible-energy ledger.

Scope: fictional project and synthetic observations. No university endorsement or real field results are claimed.

1. Research question and scope

A small energy store must trade grid draw against battery wear while handling a bounded uncertain solar input. Dispatch decisions require an explicit feasible-energy ledger. The question is deliberately narrow: can the proposed decision rule improve the stated baseline on the supplied cases while keeping every outcome inspectable? The result is a computing graduation-project demonstrator, not operational guidance. Inputs, assumptions and failure definitions are part of the assessment scope; benefits outside that scope are not inferred.

2. Contribution and alternatives

Compare greedy self-consumption, fixed reserve and dynamic-programming dispatch on seeded daily profiles; independently enumerate short horizons and perturb demand and battery bounds. The documented alternatives are: Greedy solar self-consumption; fixed battery reserve. The design favors a small auditable implementation over an opaque external model. It exposes the constraints and the reason for each rejection rather than returning only an unexplained aggregate number. A more elaborate system might fit more realistic data, but would introduce training, calibration and validation needs absent from this bounded artifact.

3. Method and reproducibility

The owned reference experiment is experiments/microgrid.py. It was executed on thirty separate seeded cases beginning at seed 53500. The same seed and project definition reproduce every CSV row. The public results file records the complete small dataset, not a hand-selected best example. The program uses Python standard-library arithmetic and deterministic random.Random inputs. No external private dataset or participant record is required. To reproduce, run the attached script with Python3.14 and compare the JSON rows with results/validation.csv. The web platform never executes this uploaded code.

4. Assumptions and decision record

The simplified battery and tariff model omits degradation chemistry, grid protection and real installation certification. No field cost saving is claimed. The model treats its input bounds as explicit preconditions. Missing or infeasible states are distinguished from successful outputs. Cost units are simulator units; they are not currency, human waiting-time estimates or certified safety margins. The baseline and proposed behavior receive the same input realization. Internal assertions check the named invariant or independent small-instance oracle before rows are emitted. A failed assertion stops generation; no successful result is synthesized to hide it.

5. Validation and evidence

The attached results contain 30 cases with 5 reported quantities. The table below is calculated from those exact rows. Minimum, maximum and mean are descriptive statistics of the supplied synthetic cases only. They do not estimate a population, establish statistical significance or measure the Capstone platform itself. Inspect individual rows before relying on the mean: unreachable routes, ties, budget limits and replay behavior are meaningful outcomes, not missing values to be silently discarded.

Reported quantity	Min	Max	Mean
no storage cost units	5	42	17.9
optimal cost units	2	35	12.83
fixed reserve cost units	2	42	16
oracle match	1	1	1
horizon steps	6	6	6

Each mean uses the valid_n denominator recorded in results/summary.json.

6. Critical interpretation

The primary value is traceability between a documented decision, executable reference and inspectable output. A positive comparison within this model does not prove superiority in deployment. The intervention changes more than one practical assumption relative to a real system: input quality, failure costs and operator behavior are simplified. Reviewers should credit reproducibility only within the declared scope, and should lower an impact or generalizability judgment when broader claims are not supported. The four independent rubric criteria remain judgments of the artifact; the simulator does not manufacture academic grades.

7. Security, privacy and operations

No personal data is used in these generated artifacts. Code and data are presented as inert native files. The platform controls public versus restricted delivery and exact frozen-version packets independently of file names. A separate restricted annex is available only through a current authorized grant or assessment packet. Research-file checksums provide byte integrity, not trust in arbitrary code. A real follow-on study would need threat modeling, input-provenance review, operational ownership and a documented failure response before real deployment.

8. Limitations and next experiment

The simplified battery and tariff model omits degradation chemistry, grid protection and real installation certification. No field cost saving is claimed. A suitable next experiment would replace one synthetic assumption at a time with independently validated observations, preserve a held-out case set, and preregister the comparison metric. Boundary and negative cases should be expanded before adding a more complex predictor. Any real participants, devices, institutions or field sites would require their own approvals and provenance. None is claimed by this demonstration.

9. Artifact and assessment map

README.md is the entry point. The research report describes question, alternatives, method, assumptions and interpretation. The experiment source supplies the exact reproducible calculation. results/validation.csv holds every case; results/summary.json holds the unrounded structured summary. design/architecture.md describes boundaries. These artifacts support evidence locators for contribution, rigor, validation and practical impact. The cover image is an AI-generated illustration of the subject and is not experiment evidence.

10. Provenance

All people, projects, assessment inputs and simulated observations in this bundle were authored for a controlled P5.3 demonstration. Real university names identify demo tenants only; no affiliation, real submission, institutional verification, academic endorsement, publication or award is asserted. No citations to unverified publications are invented. The comparison baselines are fully specified local algorithms or record-checking rules included in the artifact, so the claim can be checked without an external authority.

Appendix A. Sequential case evidence

These are the first eight cases in generation order, without selecting favorable outputs. The complete thirty-case ledger is in results/validation.csv. Units and interpretation follow the reference source; categorical identifiers are retained for traceability, not averaged.

case	no storage cost units	optimal cost units	fixed reserve cost units	oracle match	horizon steps
1	15	10	10	1	6
2	13	7	10	1	6
3	24	17	23	1	6
4	37	30	37	1	6
5	11	7	10	1	6
6	5	3	3	1	6
7	24	17	21	1	6
8	20	14	20	1	6

Audit the complete artifact

Each case invokes the named function in the attached Python file. Internal assertions execute before a row is returned. Re-running the full file must reproduce the supplied case ledger; changing a seed or precondition creates a different experiment and must be reported as such. The source is provided for inspection and authorized offline reproduction. Capstone never executes uploaded code.

Appendix B. Reference implementation

This display excerpt comes from the exact attached source. UTF-8 corpus definitions, where present, are omitted only from this Latin-font PDF listing; the complete original UTF-8 source remains available. Wrapped lines below are display continuation, not an alternative executable file.

```
def microgrid(seed):
    rng = random.Random(seed)
    net = [rng.randint(-3, 4) for _ in range(6)]
    tariffs = [1, 1, 3, 4, 2, 1]
    def search(t, energy):
        if t == 6: return 0
        return min(max(0, net[t] + delta) * tariffs[t] + search(t + 1, energy + delta)
                   for delta in [-1, 0, 1] if 0 <= energy + delta <= 3)
    states = {1: 0}
    for t, need in enumerate(net):
        nxt = {}
        for energy, cost in states.items():
            for delta in [-1, 0, 1]:
                e = energy + delta
                if 0 <= e <= 3:
                    v = cost + max(0, need + delta) * tariffs[t]
                    nxt[e] = min(nxt.get(e, math.inf), v)
        states = nxt
    optimal = min(states.values())
    assert optimal == search(0, 1)
    baseline = sum(max(0, n) * p for n, p in zip(net, tariffs))
    energy=1; fixed_reserve=0
    for need,price in zip(net,tariffs):
        delta=min(-need,3-energy,1) if need<0 else -min(need,max(0,energy-1),1)
        energy+=delta;fixed_reserve+=max(0,need+delta)*price
        assert 0<=energy<=3
    assert optimal <= baseline and optimal <= fixed_reserve
    return {'no_storage_cost_units': baseline, 'optimal_cost_units': optimal,
            'fixed_reserve_cost_units': fixed_reserve,
            'oracle_match': 1, 'horizon_steps': 6}
```